


**4AI08 - STATISTICS: LINEAR MODELS**

# Practical Work 1

## Leave-One-Out Cross-Validation

**Full Name :** **Imane SIHAMI**

Notebook and source code on GitHub

# Contents

**1 Introduction** **1**

**2 Exercise 1: Deleted Residuals** **1**

2.1 Question 1: Decomposition after deleting one observation . . . . . 1

2.2 Question 2: Relation involving the complete Gram matrix . . . . . 2

2.3 Question 3: Sherman–Morrison formula . . . . . 2

2.4 Question 4: Compact expression for the deleted estimator . . . . . 3

2.5 Question 5: Deleted-residual identity . . . . . 3

**3 Exercise 2: Coding Leave-One-Out Cross-Validation** **4**

3.1 Part 1: OLS from Scratch . . . . . 4

3.1.1 Question 1: Data, Gram matrix and invertibility . . . . . 4

3.1.2 Question 2: OLS estimator . . . . . 4

3.1.3 Question 3: Residual diagnostics and uncertainty . . . . . 5

3.1.4 Question 4: Validation with scikit-learn . . . . . 6

3.2 Part 2: Leave-One-Out Cross-Validation . . . . . 7

3.2.1 Question 1: Efficient algorithm and computational cost . . . . . 7

3.2.2 Question 2: Naive and efficient implementations . . . . . 7

3.2.3 Question 3: Running-time comparison . . . . . 9

**4 Conclusion** **10**

# 1 Introduction

We consider the fixed-design Gaussian linear model

$$\boldsymbol{y} = X\boldsymbol{\theta} + \boldsymbol{\varepsilon}, \qquad X \in \mathbb{R}^{n\times p}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_n),$$

where $X$ has full column rank. Its columns include the intercept whenever an intercept is used. The ordinary least-squares (OLS) estimator is

$$\widehat{\boldsymbol{\theta}} = (X^\top X)^{-1} X^\top \boldsymbol{y}.$$

We denote by $x_i \in \mathbb{R}^p$ the feature vector associated with observation $i$; therefore, $x_i^\top$ is the $i$-th row of $X$. This convention makes the dimensions explicit:

$$x_i^\top \in \mathbb{R}^{1\times p}, \qquad x_i \in \mathbb{R}^{p\times 1}.$$

In this practical work, we first derive a formula for the leave-one-out deleted residual. We then implement OLS from scratch and compare two LOOCV procedures: a direct method that refits the model after removing each observation, and a faster method that reuses quantities obtained from a single fit. The numerical experiment will allow us to check that both methods give the same residuals and to compare their running times.

> **Notation used throughout the report**
>
> The hat matrix and the ordinary residuals are
>
> $$H = X(X^\top X)^{-1}X^\top, \qquad \widehat{\boldsymbol{\varepsilon}} = \boldsymbol{y} - X\widehat{\boldsymbol{\theta}}.$$
>
> Its $i$-th diagonal entry is the leverage
>
> $$h_{ii} = x_i^\top (X^\top X)^{-1} x_i.$$

# 2 Exercise 1: Deleted Residuals

After deleting observation $i$, let $X_{(i)}$ and $\boldsymbol{y}_{(i)}$ denote the reduced design matrix and response vector. The corresponding estimator and deleted residual are

$$\widehat{\boldsymbol{\theta}}^{(i)} = \left(X_{(i)}^\top X_{(i)}\right)^{-1} X_{(i)}^\top \boldsymbol{y}_{(i)}, \qquad \widehat{\varepsilon}_i^{(i)} = y_i - x_i^\top \widehat{\boldsymbol{\theta}}^{(i)}.$$

The objective is to establish the shortcut

$$\boxed{\widehat{\varepsilon}_i^{(i)} = \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}}.$$

## 2.1 Question 1: Decomposition after deleting one observation

Since the rows of $X$ are $x_1^\top, \dots, x_n^\top$, the Gram matrix can be written as

$$X^\top X = \sum_{j=1}^{n} x_j x_j^\top.$$

Separating the contribution of observation $i$ gives

$$X^\top X = \sum_{j\neq i} x_j x_j^\top + x_i x_i^\top = X_{(i)}^\top X_{(i)} + x_i x_i^\top.$$

Similarly,

$$X^\top \boldsymbol{y} = \sum_{j=1}^{n} x_j y_j = \sum_{j \neq i} x_j y_j + x_i y_i = X_{(i)}^\top \boldsymbol{y}_{(i)} + x_i y_i.$$

Therefore,

**Answer to Question 1**

$$\boxed{X^\top X = X_{(i)}^\top X_{(i)} + x_i x_i^\top}, \qquad \boxed{X^\top \boldsymbol{y} = X_{(i)}^\top \boldsymbol{y}_{(i)} + x_i y_i}.$$

### 2.2 Question 2: Relation involving the complete Gram matrix

The second identity from Question 1 implies

$$X_{(i)}^\top \boldsymbol{y}_{(i)} = X^\top \boldsymbol{y} - x_i y_i.$$

Multiplying by $(X^\top X)^{-1}$ yields

$$(X^\top X)^{-1} X_{(i)}^\top \boldsymbol{y}_{(i)} = (X^\top X)^{-1} X^\top \boldsymbol{y} - (X^\top X)^{-1} x_i y_i$$

$$= \widehat{\boldsymbol{\theta}} - (X^\top X)^{-1} x_i y_i.$$

Hence,

**Answer to Question 2**

$$\boxed{(X^\top X)^{-1} X_{(i)}^\top \boldsymbol{y}_{(i)} = \widehat{\boldsymbol{\theta}} - (X^\top X)^{-1} x_i y_i}.$$

### 2.3 Question 3: Sherman–Morrison formula

From Question 1,

$$X_{(i)}^\top X_{(i)} = X^\top X - x_i x_i^\top.$$

We use the Sherman–Morrison identity

$$(A - uv^\top)^{-1} = A^{-1} + \frac{A^{-1} u v^\top A^{-1}}{1 - v^\top A^{-1} u}.$$

With $A = X^\top X$ and $u = v = x_i$, we obtain

$$\left(X_{(i)}^\top X_{(i)}\right)^{-1} = (X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - x_i^\top (X^\top X)^{-1} x_i}.$$

Using $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$ and multiplying by $X_{(i)}^\top \boldsymbol{y}_{(i)}$ gives

**Answer to Question 3**

$$\boxed{\widehat{\boldsymbol{\theta}}^{(i)} = \left[(X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - h_{ii}}\right] X_{(i)}^\top \boldsymbol{y}_{(i)}}.$$

### 2.4 Question 4: Compact expression for the deleted estimator

Let $A = (X^\top X)^{-1}$. From Question 3,

$$\widehat{\boldsymbol{\theta}}^{(i)} = \left( A + \frac{A x_i x_i^\top A}{1 - h_{ii}} \right) X_{(i)}^\top \boldsymbol{y}_{(i)}.$$

Question 2 gives

$$A X_{(i)}^\top \boldsymbol{y}_{(i)} = \widehat{\boldsymbol{\theta}} - A x_i y_i.$$

Substitution therefore leads to

$$\widehat{\boldsymbol{\theta}}^{(i)} = \widehat{\boldsymbol{\theta}} - A x_i y_i + \frac{A x_i x_i^\top (\widehat{\boldsymbol{\theta}} - A x_i y_i)}{1 - h_{ii}}$$

$$= \widehat{\boldsymbol{\theta}} - A x_i y_i + \frac{A x_i (x_i^\top \widehat{\boldsymbol{\theta}} - h_{ii} y_i)}{1 - h_{ii}}$$

$$= \widehat{\boldsymbol{\theta}} + \frac{A x_i [-y_i (1 - h_{ii}) + x_i^\top \widehat{\boldsymbol{\theta}} - h_{ii} y_i]}{1 - h_{ii}}$$

$$= \widehat{\boldsymbol{\theta}} - \frac{A x_i (y_i - x_i^\top \widehat{\boldsymbol{\theta}})}{1 - h_{ii}}.$$

Because $\widehat{\varepsilon}_i = y_i - x_i^\top \widehat{\boldsymbol{\theta}}$, we conclude that

**Answer to Question 4**

$$\boxed{\widehat{\boldsymbol{\theta}}^{(i)} = \widehat{\boldsymbol{\theta}} - \frac{(X^\top X)^{-1} x_i \widehat{\varepsilon}_i}{1 - h_{ii}}}.$$

### 2.5 Question 5: Deleted-residual identity

By definition,

$$\widehat{\varepsilon}_i^{(i)} = y_i - x_i^\top \widehat{\boldsymbol{\theta}}^{(i)}.$$

Inserting the expression from Question 4 gives

$$\widehat{\varepsilon}_i^{(i)} = y_i - x_i^\top \widehat{\boldsymbol{\theta}} + \frac{x_i^\top (X^\top X)^{-1} x_i}{1 - h_{ii}} \widehat{\varepsilon}_i$$

$$= \widehat{\varepsilon}_i + \frac{h_{ii}}{1 - h_{ii}} \widehat{\varepsilon}_i$$

$$= \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}.$$

**Main theoretical result**

Provided that the reduced OLS problem is well defined (equivalently, $1 - h_{ii} \neq 0$), the deleted residual is

$$\boxed{\widehat{\varepsilon}_i^{(i)} = \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}}.$$

Thus all LOOCV residuals can be recovered from one OLS fit.

# 3 Exercise 2: Coding Leave-One-Out Cross-Validation

## 3.1 Part 1: OLS from Scratch

For this part, we only use NumPy and the normal equations; no regression library is involved. We keep the random seed fixed so that the experiment can be reproduced and the numerical results remain the same from one run to the next.

### 3.1.1 Question 1: Data, Gram matrix and invertibility

The data contain $n = 50$ observations and 10 generated features. Because the generator adds an intercept column, $X$ has $p = 11$ columns.

**Listing 1:** Generation of the data and analysis of the Gram matrix.

```
X, y, theta_star = gen_data(
    n_samples=50,
    n_features=10,
    return_coef=True,
    random_state=0
)
gram_matrix = X.T @ X
eigenvalues = np.linalg.eigvalsh(gram_matrix)
print("Shape of X:", X.shape)
print("Eigenvalues:", eigenvalues)
print("Smallest eigenvalue:", np.min(eigenvalues))
print("Rank of X:", np.linalg.matrix_rank(X))
print("Invertible:", np.all(eigenvalues > 0))
```

The eigenvalues obtained are

$$(11.9020,\ 29.2139,\ 32.1271,\ 39.1982,\ 40.9985,\ 42.5345,$$

$$51.2980,\ 64.2396,\ 66.9494,\ 80.1866,\ 89.8322).$$

**Numerical conclusion**

$$X \in \mathbb{R}^{50\times 11}, \qquad \operatorname{rank}(X) = 11, \qquad \lambda_{\min}(X^\top X) = 11.9020 > 0.$$

The smallest eigenvalue is strictly positive and the rank of $X$ is equal to the number of its columns. Hence the columns are linearly independent and $X^\top X$ is invertible. We can therefore solve the normal equations and obtain a unique OLS estimate.

### 3.1.2 Question 2: OLS estimator

The normal equations are

$$X^\top X \widehat{\boldsymbol{\theta}} = X^\top \boldsymbol{y}.$$

Numerically, solving this linear system is preferable to explicitly computing $(X^\top X)^{-1}$.

**Listing 2:** OLS implementation based on the normal equations.

```
def ols_fit(X, y):
    """Return the OLS estimate theta_hat, with shape (p,)."""
    gram_matrix = X.T @ X
    right_hand_side = X.T @ y
    return np.linalg.solve(gram_matrix, right_hand_side)
theta_hat = ols_fit(X, y)
estimation_error = theta_hat - theta_star
print("Error norm:", np.linalg.norm(estimation_error))
```

Table 1 compares the coefficients used to generate the data with the values recovered by OLS. They are close, although they do not coincide exactly because Gaussian noise was added to the response.

**Table 1:** True parameters, OLS estimates and estimated standard errors.

| Index | Interpretation | $\theta_j^\star$ | $\widehat{\theta}_j$ | Standard error |
|---|---|---|---|---|
| 0 | Intercept | 7.0000 | 6.1805 | 0.8864 |
| 1 | Feature 1 | 40.3793 | 39.4008 | 0.8623 |
| 2 | Feature 2 | 85.7125 | 85.9969 | 0.8064 |
| 3 | Feature 3 | 27.1252 | 27.9865 | 0.8438 |
| 4 | Feature 4 | 39.9812 | 39.7177 | 0.7607 |
| 5 | Feature 5 | 67.1383 | 67.8228 | 0.8475 |
| 6 | Feature 6 | 11.7316 | 10.6568 | 0.8348 |
| 7 | Feature 7 | 97.9962 | 100.0275 | 0.9856 |
| 8 | Feature 8 | 42.3706 | 43.4976 | 0.8336 |
| 9 | Feature 9 | 45.7059 | 45.5795 | 0.8552 |
| 10 | Feature 10 | 34.4718 | 34.2841 | 1.1690 |

The Euclidean norm of the estimation error is

$$\|\widehat{\boldsymbol{\theta}} - \boldsymbol{\theta}^\star\|_2 = 3.0971.$$

### 3.1.3 Question 3: Residual diagnostics and uncertainty

The fitted values, residuals and unbiased noise-variance estimator are

$$\widehat{\boldsymbol{y}} = X\widehat{\boldsymbol{\theta}}, \qquad \widehat{\boldsymbol{\varepsilon}} = \boldsymbol{y} - \widehat{\boldsymbol{y}}, \qquad \widehat{\sigma}^2 = \frac{\|\widehat{\boldsymbol{\varepsilon}}\|_2^2}{n-p}.$$

The estimated covariance matrix and coefficient standard errors are

$$\widehat{\mathrm{Cov}}(\widehat{\boldsymbol{\theta}}) = \widehat{\sigma}^2 (X^\top X)^{-1}, \qquad \mathrm{SE}(\widehat{\theta}_j) = \sqrt{\widehat{\sigma}^2 [(X^\top X)^{-1}]_{jj}}.$$

**Listing 3:** Residuals, variance estimate, standard errors and coefficient of determination.

```
n, p = X.shape
y_hat = X @ theta_hat
residuals = y - y_hat
RSS = residuals @ residuals
sigma2_hat = RSS / (n - p)
gram_matrix_inverse = np.linalg.inv(X.T @ X)
covariance_theta_hat = sigma2_hat * gram_matrix_inverse
standard_errors = np.sqrt(np.diag(covariance_theta_hat))
TSS = np.sum((y - np.mean(y))  2)
R2 = 1 - RSS / TSS
orthogonality = X.T @ residuals
print("sigma^2:", sigma2_hat)
print("R^2:", R2)
print("||X^T residuals||:", np.linalg.norm(orthogonality))
```

**Table 2:** Principal OLS diagnostic quantities.

| Quantity | Numerical value |
|---|---|
| Estimated variance $\widehat{\sigma}^2$ | 29.1843 |
| Estimated standard deviation $\widehat{\sigma}$ | 5.4022 |
| Coefficient of determination $R^2$ | 0.999342 |
| $\|X^\top \widehat{\boldsymbol{\varepsilon}}\|_2$ | $2.79 \times 10^{-12}$ |

The noise used by the generator has standard deviation 5, so its true variance is 25. We obtain $\widehat{\sigma}^2 = 29.1843$. The difference is not surprising with only 50 observations. We also obtain an $R^2$ very close to one, which is consistent with the strong linear signal in the simulated data. Finally, the normal equations imply

$$X^\top X\widehat{\boldsymbol{\theta}} = X^\top \boldsymbol{y} \quad \Longrightarrow \quad X^\top(\boldsymbol{y} - X\widehat{\boldsymbol{\theta}}) = X^\top \widehat{\boldsymbol{\varepsilon}} = \mathbf{0}.$$

Numerically, we find $\|X^\top \widehat{\boldsymbol{\varepsilon}}\|_2 = 2.79 \times 10^{-12}$. This is not exactly zero because of floating-point arithmetic, but it is small enough to confirm the expected orthogonality between the residual vector and the column space of $X$.

### 3.1.4 Question 4: Validation with scikit-learn

Because $X$ already contains a column of ones, the option `fit_intercept=False` must be used.

**Listing 4:** Comparison with scikit-learn.

```
model = LinearRegression(fit_intercept=False)
model.fit(X, y)
theta_sklearn = model.coef_
R2_sklearn = model.score(X, y)
print(np.max(np.abs(theta_hat - theta_sklearn)))
print(np.isclose(R2, R2_sklearn))
```

**Validation of the OLS implementation**

The maximum absolute difference between the two coefficient vectors is

$$9.95 \times 10^{-14},$$

and both implementations return

$$R^2 = 0.9993416548.$$

This difference is negligible and comes from numerical rounding. The agreement with scikit-learn gives us a useful check of our implementation.

## 3.2 Part 2: Leave-One-Out Cross-Validation

### 3.2.1 Question 1: Efficient algorithm and computational cost

The naive procedure deletes one observation, fits OLS on the remaining sample, and repeats this operation for every $i$. Its deleted residual is

$$\widehat{\varepsilon}_i^{(i)} = y_i - x_i^\top \widehat{\boldsymbol{\theta}}^{(i)}.$$

The analytical result of Exercise 1 allows the same residual to be computed as

$$\widehat{\varepsilon}_i^{(i)} = \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}, \qquad h_{ii} = x_i^\top (X^\top X)^{-1} x_i.$$

**Efficient LOOCV algorithm**

1. Fit OLS once on the complete dataset.
2. Compute the ordinary residuals $\widehat{\boldsymbol{\varepsilon}} = \boldsymbol{y} - X\widehat{\boldsymbol{\theta}}$.
3. Compute all leverage values $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$.
4. Return $\widehat{\varepsilon}_i/(1 - h_{ii})$ for every observation.

For one OLS fit, forming $X^\top X$ costs $O(np^2)$ and solving the $p \times p$ system costs $O(p^3)$. The resulting complexities are summarised in Table 3.

**Table 3:** Approximate computational costs of the two LOOCV methods.

| Method | Number of OLS fits | Approximate cost |
|---|---|---|
| Naive LOOCV | $n$ | $O(n^2p^2 + np^3)$ |
| Efficient LOOCV | 1 | $O(np^2 + p^3)$ |

### 3.2.2 Question 2: Naive and efficient implementations

We now implement the two approaches described above. In the naive method, the model is fitted again after removing observation $i$:

$$\widehat{\boldsymbol{\theta}}^{(i)} = \left(X_{(i)}^\top X_{(i)}\right)^{-1} X_{(i)}^\top \boldsymbol{y}_{(i)}, \qquad \widehat{\varepsilon}_i^{(i)} = y_i - x_i^\top \widehat{\boldsymbol{\theta}}^{(i)}.$$

For the efficient method,

$$\widehat{\varepsilon}_i^{(i)} = \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}.$$

**Listing 5:** Naive and efficient LOOCV implementations.

```
def naive_loocv(X, y):
    """Return the n deleted residuals, refitting OLS n times."""
    n = X.shape[0]
    deleted_residuals = np.empty(n)
    for i in range(n):
        keep = np.ones(n, dtype=bool)
        keep[i] = False
        X_train = X[keep]
        y_train = y[keep]
        theta_i = ols_fit(X_train, y_train)
        y_prediction_i = X[i] @ theta_i
        deleted_residuals[i] = y[i] - y_prediction_i
    return deleted_residuals
def efficient_loocv(X, y):
    """Return the n deleted residuals from a single OLS fit."""
    theta_hat = ols_fit(X, y)
    residuals = y - X @ theta_hat
    gram_matrix_inverse = np.linalg.inv(X.T @ X)
    leverages = np.sum(
        (X @ gram_matrix_inverse) * X,
        axis=1
    )
    return residuals / (1 - leverages)
```

For the efficient method, we only need the diagonal of the hat matrix. The vectorised expression computes these leverage values directly, without constructing the full $n \times n$ matrix $H$. Since row $i$ of `X` is $x_i^\top$, its $i$-th result is exactly $x_i^\top (X^\top X)^{-1} x_i$.

**Listing 6:** Numerical comparison of the two methods.

```
loo_naive = naive_loocv(X, y)
loo_efficient = efficient_loocv(X, y)
maximum_difference = np.max(
    np.abs(loo_naive - loo_efficient)
)
loocv_mse = np.mean(loo_efficient  2)
print("Maximum difference:", maximum_difference)
print("Methods agree:", np.allclose(loo_naive, loo_efficient))
print("LOOCV MSE:", loocv_mse)
```

### LOOCV verification

The maximum absolute difference is

$$2.97 \times 10^{-13},$$

which confirms that both implementations produce the same deleted residuals, apart from numerical rounding. The mean squared leave-one-out error is

$$\mathrm{MSE}_{\mathrm{LOOCV}} = \frac{1}{n} \sum_{i=1}^{n} \left( \widehat{\varepsilon}_i^{(i)} \right)^2 = 38.1226.$$

### 3.2.3 Question 3: Running-time comparison

The experiment uses 50 generated features, hence $p = 51$ after adding the intercept. For each sample size, both implementations are run ten times and the mean and standard deviation of the elapsed times are recorded.

**Listing 7:** Benchmark over increasing sample sizes.

```
ns = [100, 200, 400, 800, 1600]
n_features = 50
n_repeats = 10
naive_mean_times, efficient_mean_times = [], []
naive_std_times, efficient_std_times = [], []
for n in ns:
    X_n, y_n = gen_data(
        n_samples=n,
        n_features=n_features,
        random_state=0
    )
    naive_times = []
    efficient_times = []
    for _ in range(n_repeats):
        start = time.perf_counter()
        naive_loocv(X_n, y_n)
        naive_times.append(time.perf_counter() - start)
        start = time.perf_counter()
        efficient_loocv(X_n, y_n)
        efficient_times.append(time.perf_counter() - start)
    naive_mean_times.append(np.mean(naive_times))
    efficient_mean_times.append(np.mean(efficient_times))
    naive_std_times.append(np.std(naive_times))
    efficient_std_times.append(np.std(efficient_times))
```

**Table 4:** Mean running times over ten repetitions.

| $n$ | Naive mean (s) | Efficient mean (s) | Naive std. (s) | Efficient std. (s) |
|---|---|---|---|---|
| 100 | 0.003852 | 0.000124 | 0.000282 | 0.000019 |
| 200 | 0.009612 | 0.000153 | 0.000115 | 0.000010 |
| 400 | 0.028347 | 0.000442 | 0.000411 | 0.000115 |
| 800 | 0.098126 | 0.000624 | 0.002328 | 0.000062 |
| 1600 | 0.359721 | 0.000962 | 0.006381 | 0.000071 |

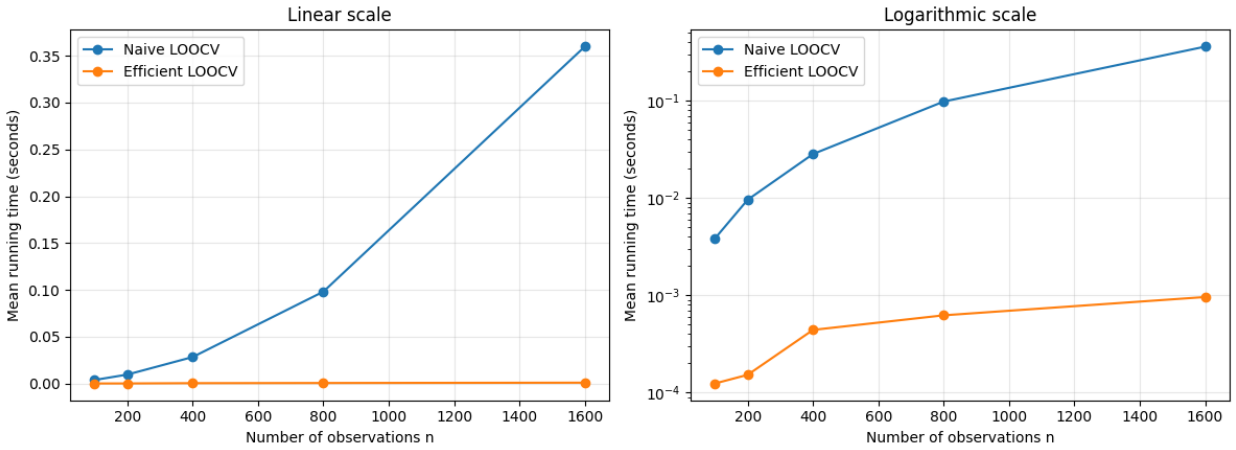



**Figure 1:** Mean running times of the naive and efficient LOOCV implementations. The left panel uses a linear scale and the right panel uses a logarithmic scale.

The difference between the two curves is already visible for small datasets and becomes much larger as $n$ increases. This is expected: the naive method fits $n$ separate models, whereas the efficient method fits only one. In our experiment, when $n = 1600$, the measured times are about 0.360 seconds and 0.000962 seconds respectively. The efficient implementation is therefore roughly 374 times faster for this sample size.

## 4 Conclusion

In the first part, we implemented OLS directly from the normal equations. The results agree with scikit-learn, and the numerical check $X^\top \widehat{\boldsymbol{\varepsilon}} \approx \mathbf{0}$ confirms the expected orthogonality of the residuals. The main theoretical identity is

$$\boxed{\widehat{\varepsilon}_i^{(i)} = \frac{\widehat{\varepsilon}_i}{1 - h_{ii}}}.$$

This formula is the main point of the TP. It gives the same deleted residuals as the naive procedure without fitting the model $n$ times. In our numerical test, the difference between the two implementations is around $3 \times 10^{-13}$, so they agree up to machine precision. The timing experiment then shows the practical benefit of the formula: the larger the dataset, the more time is saved by reusing the residuals and leverage values from the full OLS fit.